

08 — Operatoren

Rechnen mit Werten – und mit Wahrheit

HTW Berlin – Wirtschaftsingenieurwesen

SoSe 2026

Agenda

1. Wozu Operatoren?
2. Arithmetik formal: `//` und `%` als Paar
3. Vergleiche liefern **Werte**
4. Boolesche Algebra: `and` / `or` / `not` als Rechensystem
5. Präzedenz, Short-Circuit, De Morgan, XOR $\wedge$
6. `is` vs. `==` · `in` / `not in` · `+=`
7. Mengenoperationen auf Sets
8. **Klassiker: Schaltjahr** – live programmiert

Lernziel: Operatoren nicht nur anwenden, sondern als **Rechensystem** verstehen – jeder Vergleich ist ein Wert, mit dem man weiterrechnen kann.

Heute: nach jedem Konzept *Live-Demo* (ich) und *Sofort ausprobieren* (Sie, Notebook 08).

Wozu Operatoren?

Ohne Operatoren

```
print("Bestellung: Eco-Sneaker + Hemp-High")  
print("Versandfrei? Bitte im Kopf rechnen...")
```

Mit Operatoren

```
summe = 89.95 + 109.00           # 198.95  
versandfrei = summe >= 100       # True  
print(f"Versandfrei: {versandfrei}")
```

Operatoren verknüpfen Werte zu neuen Werten: Zahlen zu Zahlen (+), Zahlen zu **Wahrheit** (>=), Wahrheit zu Wahrheit (and). Heute schauen wir zum ersten Mal formal hin.

Arithmetik - bekannt aus NB 05, heute formal

Operator	Bedeutung	Beispiel
+ / -	Addition / Subtraktion	89.95 + 109.00
*	Multiplikation	89.95 * 3
/	Division (immer float)	100 / 3 $\rightarrow$ 33.33...
//	Ganzzahl-Division	100 // 3 $\rightarrow$ 33
%	Modulo (Rest)	100 % 3 $\rightarrow$ 1
**	Potenz	2 ** 10 $\rightarrow$ 1024

► **Live-Demo** - 01_arithmetik_preise.py

Unter der Haube: // und % sind ein Paar

```
bestellte = 47           # Artikel bestellt
karton     = 12          # passen in einen Karton
print(bestellte // karton) # 3   volle Kartons
print(bestellte % karton)  # 11  bleiben einzeln
```

Division mit Rest – eine Gleichung verbindet beide:

$$a = b \cdot \left\lfloor \frac{a}{b} \right\rfloor + (a \bmod b) \quad \text{Probe: } 47 = 12 \cdot 3 + 11$$

// beantwortet “wie oft passt es ganz?”, % “was bleibt übrig?” – zusammen rekonstruieren sie die Zahl exakt. Deshalb prüft `zahl % n == 0` Teilbarkeit – das braucht unser Klassiker heute.

Vergleichsoperatoren

Operator	Bedeutung	Beispiel
<code>== / !=</code>	gleich / ungleich	<code>bestand == 0</code>
<code>< / ></code>	kleiner / größer	<code>summe > 100</code>
<code><= / >=</code>	kleiner-gleich / größer-gleich	<code>summe >= 100</code>

- Ergebnis ist **immer** True oder False – Typ bool
- Beide Seiten dürfen Ausdrücke sein: `netto * 1.19 >= 100`

Fehlerklassiker = vs. `==`: `preis = 100` **setzt** den Wert (Zuweisung), `preis == 100` **fragt** nach ihm (Vergleich).

Unter der Haube: Vergleiche liefern Werte

```
preis = 135.50           # Bambus-Boot
teuer = preis > 100       # 1) auswerten -> True
print(teuer)             # 2) der Wert lebt weiter:
print(f"Premium? {teuer}") # speichern, ausgeben, ...
```

- `preis > 100` ist ein **Ausdruck**: Python wertet ihn zu einem bool-**Wert** aus – genau wie `2 + 3` zu 5
- Speichern, kombinieren, per `return` zurückgeben – **Bedingung** und **Reagieren** (NB 09) sind entkoppelt

► **Live-Demo** – 02_vergleich_bestand.py

✎ **Sofort ausprobieren** – Notebook 08, Kap. 2: alle sechs Vergleiche

Logische Operatoren - and, or, not

```
summe          = 89.95           # Eco-Sneaker
stammkunde     = True            # Anna Mueller
storniert      = False
versandfrei    = summe >= 100 or stammkunde    # True
gueltig        = not storniert and summe > 0   # True
```

- and: **beide** wahr · or: **mindestens eine** wahr · not: kippt den Wert
- Die Operanden sind bool-Werte – gern direkt aus Vergleichen

► **Live-Demo** – 03_logik_versandfrei.py

✎ **Sofort ausprobieren** – Notebook 08, Kap. 3: Party-Bedingung (and), Museum kostenlos (or), not-Operator

Unter der Haube: Boolesche Algebra als Rechensystem

Mit Wahrheitswerten kann man **rechnen** – die vollständige Tabelle *ist* die Definition:

a	b	a and b	a or b	not a
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Rückbezug E4: and / or / not **sind** die AND-/OR-/NOT-Gatter aus der Hardware – Ihre CPU besteht aus Milliarden davon. Boolesche Algebra (George Boole, 1847) ist das Rechensystem der Schaltkreise – und ab heute auch Ihres.

Unter der Haube: Auswertungsreihenfolge als Trace

Vorrang (stark → schwach): () → Arithmetik → Vergleiche → not → and → or

a, b, c = True, True, False

ergebnis = not a or b and c *# was passiert?*

Schritt	Python rechnet	Wert
1	not a (not zuerst)	False
2	b and c (and vor or)	False
3	False or False	False

Merken müssen Sie nur: **not vor and vor or**. Alles Weitere: Klammern – (not a) or (b and c) rechnet identisch, liest sich aber sofort.

Unter der Haube: Short-Circuit - faule Auswertung

```
bestand = 0  
check = bestand != 0 and 100 / bestand > 5  
print(check)      # False -- Division NIE ausgeführt
```

Ausdruck	links	rechts	Ergebnis
False and X	False	wird übersprungen	False
True or X	True	wird übersprungen	True

and / or werten das zweite Argument nur aus, wenn es das Ergebnis noch **ändern kann**. So *schützt* die linke Prüfung (bestand != 0) den rechten Ausdruck – dieses Muster sehen Sie ständig wieder.

De-Morgan-Gesetze - Verneinung verteilen

Alltag: “NICHT (vegan **und** lieferbar)” bedeutet: “nicht vegan **oder** nicht lieferbar.”

```
vegan, lieferbar = True, False
problem_a = not (vegan and lieferbar)      # True
problem_b = not vegan or not lieferbar     # True
```

De Morgan: $\text{not } (a \text{ and } b) = \text{not } a \text{ or } \text{not } b$ und $\text{not } (a \text{ or } b) = \text{not } a \text{ and } \text{not } b$. Beim Hineinziehen von not kippt and $\leftrightarrow$ or.

Unter der Haube: De Morgan beweisen - und mit Python prüfen

a	b	not (a and b)	not a or not b
True	True	False	False
True	False	True	True
False	True	True	True
False	False	True	True

```
a, b = True, False
links  = not (a and b)
rechts = not a or not b
print(links == rechts)    # True (alle 4 Faelle!)
```

Zwei Ausdrücke sind **logisch gleich**, wenn ihre Wahrheitstabellen Zeile für Zeile übereinstimmen - die Tabelle ist der Beweis, Python die Stichprobe. Genau diesen Check nutzen wir gleich beim Klassiker.

XOR ^ - entweder-oder (exklusiv)

```
code_post = True      # Rabattcode per Post  
code_app  = True      # Rabattcode in der App  
einloesbar = code_post ^ code_app  
print(einloesbar)     # False -- nur EIN Code zaehlt
```

a	b	a ^ b
True	True	False
True	False	True
False	True	True
False	False	False

► **Live-Demo** - 06_boolesche_algebra.py (Wahrheitstabellen, De Morgan, XOR - auch als Formel: $a \wedge b = (a \vee b) \wedge \text{not } (a \wedge b)$)

== vs. is - Wert vs. Identität

```
a = [1, 2, 3]
b = a                                # 2. Etikett, gleiches Objekt
c = [1, 2, 3]                        # neues Objekt, gleicher Inhalt
print(a == c, a is c)               # True False
print(a == b, a is b)               # True True
```

- Rückbezug NB 04: Namen sind **Etiketten an Objekten** - == vergleicht den **Inhalt**, is fragt: *ist es dasselbe Objekt im Speicher?*
- Praxis: is fast nur für is None / is not None - Werte vergleichen
Sie **immer** mit ==

► **Live-Demo** - 05_identitaet_vs_gleichheit.py

in / not in - Zugehörigkeit

```
warenkorb = ["Eco-Sneaker", "Hemp-High"]  
print("Eco-Sneaker" in warenkorb)          # True  
print("Bambus-Boot" not in warenkorb)      # True  
print("eco" in "Eco-Sneaker".lower())     # True
```

- Funktioniert auf Listen, Tupeln, Sets, Dicts (prüft **Schlüssel!**) und Strings (prüft **Teilstrings**)
- Unter der Haube: Python vergleicht Element für Element mit == – das Ergebnis ist wieder ein bool-Wert

► **Live-Demo** – 04_zugehoerigkeit_warenkorb.py

📝 **Sofort ausprobieren** – Notebook 08, Kap. 4: is-Operator, in mit Lieblingszahlen, not in mit Benutzernamen, pruefe_liste-Funktion

Zuweisungs-Operatoren - += und Familie

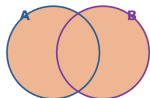
```
lager = 100
lager += 45      # Python: lager = lager + 45
lager -= 12      # Python: lager = lager - 12
print(lager)     # 133
```

Kurzform	Python führt aus
$x += n$ / $x -= n$	$x = x + n$ / $x = x - n$
$x *= n$ / $x /= n$	$x = x * n$ / $x = x / n$
$x //= n$, $x \% = n$, $x **= n$	analog

Unter der Haube dieselbe Mechanik wie in NB 04: **rechts zuerst** mit dem *alten* Wert rechnen, dann den Namen neu binden. += ist Abkürzung, kein neues Konzept.

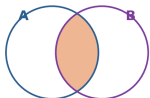
Mengenoperationen auf Sets

Vereinigung · $A \cup B$



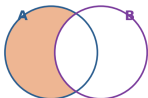
$A \cup B$

Schnittmenge · $A \cap B$



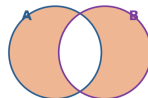
$A \cap B$

Differenz · $A - B$



$A \setminus B$

Symm. Differenz · $A \triangle B$



$A \triangle B$

$$A \cup B = A \cup B$$

$$A \cap B = A \cap B$$

$$A - B = A \setminus B$$

$$A \triangle B = A \triangle B$$

alles aus beiden

nur Gemeinsames

nur in A

in **genau einer**

$\wedge$ kennen Sie seit eben: die symmetrische Differenz ist **XOR auf Mengen** – ein Element gehört zu $A \triangle B$, wenn es in *genau einer* der beiden Mengen liegt.

Klassiker: Schaltjahr - die Aufgabe

Der Klassiker jeder Programmierschule – Pointe: Sie brauchen kein `if` –
der Ausdruck IST die Entscheidung.

- Ein Jahr ist Schaltjahr, wenn es durch 4 teilbar ist –
- **außer** es ist durch 100 teilbar –
- **außer** es ist *doch* durch 400 teilbar.
- Schreiben Sie `ist_schaltjahr(jahr)` – gibt `True` / `False` zurück;
testen Sie mit 2024 und 1900

So soll es laufen

Jahr: 2024

Schaltjahr? `True`

Jahr: 1900

Schaltjahr? `False`

Klassiker: Schaltjahr - Live-Coding

► Live-Coding ab leerem Editor

Referenz: 90_klassiker_schaltjahr.py - Teilziele:

1. Die Regel in Worte fassen: durch 4 - außer durch 100 - außer doch durch 400
2. Teilbarkeit ausdrücken: `jahr % 4 == 0`
3. Teilregeln mit and / or und **Klammern** verknüpfen
4. De-Morgan-Check: alternative Formulierung, gleiche Wahrheitstabelle
5. Als Funktion verpacken: `def ist_schaltjahr(jahr): return ...`

 **Zum Selberlösen** - Handout *"Klassiker 08: Schaltjahr"* (Klassiker/08_Klassiker_Schaltjahr.pdf). Vertiefung: Notebook 08, Trainingsmaterial.

Cheat Card

Aufgabe	Syntax
Ganzzahl-Division / Rest	<code>a // b</code> , <code>a % b</code> (teilbar: <code>a % n == 0</code>)
Vergleich (ergibt bool)	<code>==</code> <code>!=</code> <code><</code> <code>></code> <code><=</code> <code>>=</code>
Logik (Vorrang: not, and, or)	<code>and</code> , <code>or</code> , <code>not</code>
Entweder-oder (XOR)	<code>a ^ b</code>
De Morgan	<code>not (a and b) = not a or not b</code>
Wert vs. Objekt	<code>==</code> vs. <code>is</code> (nur <code>is None</code>)
Enthalten?	<code>x in s</code> , <code>x not in s</code>
Kurz-Zuweisung	<code>x += 1</code> , <code>x -= 1</code> , ...
Klassiker heute	Schaltjahr: ein boolescher Ausdruck

Faustregel: Jeder Vergleich ist ein **Wert**. Wer Wahrheit als Wert begreift, kann mit ihr rechnen – speichern, kombinieren, zurückgeben.

Ausblick: Notebook 09 - Verzweigungen

Unser `ist_schaltjahr(2024)` liefert `True` – aber das Programm kann darauf noch nicht **reagieren**: es druckt den Wert nur.

Nächste Woche werden aus Werten **Wege** (Syntax ab NB 09 – heute nur ein Ausblick):

```
if ist_schaltjahr(jahr):  
    print("366 Tage -- der Februar hat 29.")  
else:  
    print("365 Tage.")
```

Die Bedingung hinter `if` ist **exakt** so ein boolescher Ausdruck wie heute – alles, was zwischen den Klammern zählt, können Sie schon.

Heute geübt

- ✓ // und % als Paar: Division mit Rest, Teilbarkeit via $\% == 0$
- ✓ Vergleiche liefern bool-**Werte** – speicherbar, kombinierbar
- ✓ Boolesche Algebra: Wahrheitstabellen als Rechensystem (= E4-Gatter)
- ✓ Vorrang not $\rightarrow$ and $\rightarrow$ or – und Short-Circuit
- ✓ De Morgan: not verteilen, and $\leftrightarrow$ or kippt
- ✓ XOR ^, is vs. ==, in / not in, +=
- ✓ Mengenoperationen auf Sets: |, &, -, ^
- ✓ **Klassiker Schaltjahr**: ein Ausdruck, keine Verzweigung

Ihre Aufgaben bis zur nächsten Woche:

- Klassiker **Schaltjahr** eigenständig lösen (Handout) – erst danach in die Musterlösung schauen
- “Sofort ausprobieren”-Aufgaben im Notebook 08
- Trainingsmaterial: `ist_gerade`, `ist_im_bereich`, `passwort_stark`